MM/GBSA and MM/PBSA calculation based on single PDB files by MolAICal

Qifeng Bai
Email: molaical@yeah.net
Homepage: https://molaical.github.io or https://molaical.gitlab.io
School of Basic Medical Sciences
Lanzhou University
Lanzhou, Gansu 730000, P. R. China

1. Introduction

In this tutorial, the MolAICal (https://molaical.github.io) is used to calculate the MM/GBSA and MM/PBSA between ligand N3 and SARS-CoV-2 Mpro based on a single PDB file. This tutorial can **not only** be used to calculate MM/GBSA or MM/PBSA of protein-ligand **but also** MM/GBSA or MM/PBSA of protein-peptide, protein-protein, DNA-ligand, DNA-protein, protein-DNA, protein-RNA, and any complex based on a single PDB file. It just replaces the protein and ligand of this tutorial with the appointed objects. For example, **protein** in this tutorial is substituted for **DNA**, and **ligand** in this tutorial is substituted for **peptide based on the same running command arguments of this tutorial**.

2. Materials

2.1. Software requirement

1) MolAICal: https://molaical.github.io

2) NAMD (CPU version, e.g., "Linux-x86_64-multicore"): https://www.ks.uiuc.edu/Research/namd/ (**Notice:** This tutorial can be run by **NAMD 2.x, 3.x, or a higher version.** For example, the command "**namd3**" substitutes for the command "**namd2**" in this tutorial **if you use NAMD 3.x version**. For a higher version of NAMD, users can use a similar replacement way as the previous example.)

3) Carma: https://github.com/glykos/carma or https://utopia.duth.gr/~glykos/Carma.html

4) APBS (https://apbs.readthedocs.io) or DelPhi (http://compbio.clemson.edu/lab/delphisw)

2.2. Example files

1) All the necessary tutorial files are downloaded from:
https://github.com/MolAICal/tutorials/tree/master/021-single_mmpbgbsa

3. Procedure

To address the library dependency issues in Linux, **the Linux version of MolAICal (Not for Windows version)** adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
```

```
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal

container.
To prevent the loss of installed programs (e.g., VMD and NAMD) when rebuilding the MolAICal image, refer to step 3 in **Appendix 1**.

Assuming users have configured the internal commands in MolAICal for VMD and NAMD using the following instructions:

```
#> molaical.exe -call set -n VMD -p "e:/soft/vmd193/bin/vmd"
#> molaical.exe -call set -n namd -p "e:/soft/namdcpu/namd3"
#> molaical.exe -call set -n delphi -p "e:/soft/delphicpp_v8.4.3_serial"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. When running MMPB/GB/SA functionality, and setting environment variables using MolAICal (e.g., #> molaical.exe -call set -n VMD -p "path"), the variable names in MMPB/GB/SA corresponding to '-n' are fixed: 'vmd', 'namd', and 'delphi' (case insensitive).

- ✧ DelPhi is a binary file and requires executable permission on Linux systems: " chmod +x delphicpp_v8.4.3_serial ". Therefore, users can refer to the NAMD installation steps in Appendix 1 or the above steps.
- ✧ Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

To ensure the reproducibility of MM/PBSA and MM/GBSA results, the following conditions must be strictly met:

- ✧ Use the **CPU version of NAMD**, such as "Linux-x86_64-multicore";
- ✧ **NAMD** must employ **only 1 CPU core** for computation;
- ✧ If using a custom NAMD input file (e.g., "md_configure.conf" in this example), a fixed random seed must be set, such as **"seed 12345"** in "md_configure.conf".

3.1. Found a problem

Go to the tutorial directory:
#> cd 021-single_mmpbgbsa

The "complex.pdb" is a file containing protein and ligand. To calculate MM/GBSA or MM/PBSA, it needs to get the PSF format file of "complex.pdb" by Autopsf of VMD. I found Autopsf of VMD has a bug for loading topology, so it needs to fix the bug in Autopsf of VMD as follows:

```
#> molaical.exe -call run -c fixpsf
```

Then call VMD to generate the PSF format file of "complex.pdb" by MolAICal:

```
#> molaical.exe -call run -c vmdargs -i -pdb complex.pdb -dispdev text -args autopsf autopsf -mol 0 -top ligand.str -prefix complex
```

Note: If running VMD "**autopsf**", it should fix the topology bug of VMD by MolAICal. Please see

the above related introduction. The arguments after '-i' are the VMD parameters. The detailed information of the arguments is as follows:

'-pdb': means to load the molecule in PDB format.

'-dispdev text': means to load VMD in text mode.

'-args': pass subsequent arguments to the text interpreter to run the VMD command once in the Tk console:

- The first parameter after **'-args'** is a package name that needs to be loaded. For example, "package require **autopsf**"
- The second parameter after **'-args'** is a command after loading the corresponding package. And the following strings are the arguments of this command. For example, "**autopsf** autopsf -mol 0 -top ligand.str"
- Autopsf defaults to load the topology files containing "top_all36_prot.rtf", "top_all36_lipid.rtf", "top_all36_na.rtf", "top_all36_carb.rtf", "top_all36_cgenff.rtf", "toppar_all36_carb_glycopeptide.str", "toppar_water_ions_namd.str". Therefore, users can refer to these topology files. If there's no need to introduce special topology files, they can avoid the hassle of inputting "-top".

✧ Options: users can check the help information of "**autopsf**" of VMD as follows:

```
#> molaical.exe -call run -c vmdargs -i -dispdev text -args autopsf autopsf
```

At last, it will generate "complex_formatted_autopsf.pdb" and "complex_formatted_autopsf.psf". Now, calculate the MM/GBSA as follows:

```
#> molaical.exe -call run -c mmpbgbsa1 -i -dispdev text -args -s complex_formatted_autopsf.psf -c complex_formatted_autopsf.pdb -cs "segname,AP1,CO1" -rs "segname,AP1" -ls "segname,CO1" -pb 0 -gb 1 -ff ligand.str
```

Note: Here, MolAICal supports recognizing commas "," in strings like "segname,AP1,CO1" as equivalent to spaces " ", meaning "segname,AP1,CO1" is treated identically to "segname AP1 CO1". This applies to similar characters by analogy. This input method helps prevent command-line parsing errors caused by spaces.

It will get: **G binding: 60422.8572 kcal/mol.**

Clearly, without performing energy minimization or molecular dynamics simulations, predictions of non-covalent interactions are highly inaccurate. As such, MolAICal provides a methodology based on energy minimization and molecular simulations for calculating MM/GBSA and MM/PBSA.

3.2. Calculate MM/GBSA based on a complex file in the Windows system

Run command below can calculate MM/GBSA for "complex.pdb"

```
#> molaical.exe -call run -c script -n runScript_win.dat -i complex.pdb ligand.str complex
```

Note:

- ✧ MolAICal defaults to providing molecular dynamics simulations with the Generalized Born Implicit Solvent model, where parameters can be directly modified via command-line arguments as following parameter introduction such as "-system_topology" and "-system_coords".
- ✧ The strings after '-i' are the parameter list that will be split by space. The split strings will replace the strings $molargs1, $molargs2, $molargs3, etc. in the script file. The number of the string $molargs1, $molargs2, $molargs3, etc., is the same as the length of the split strings after '-i'. Please see script file named "runScript_win.dat":

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs1 -dispdev text -args autopsf autopsf -mol 0 -top $molargs2 -prefix $molargs3

# 3. default to run 20 fs minimiztion and 40 fs MD simulation
molaical.exe -call run -c runnamd -i -dispdev text -args -s $molargs3_formatted_autopsf.psf -c $molargs3_formatted_autopsf.pdb -ff $molargs2

# 4. Cut last 10 steps for MM/GBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out result.dcd -first 51 -last 60 -stride 1 $molargs3_formatted_autopsf_md.dcd

# 5. Calculate MM/GBSA
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s $molargs3_formatted_autopsf.psf -c $molargs3_formatted_autopsf.pdb -t result.dcd -cs "segname,AP1,CO1" -rs "segname,AP1" -ls "segname,CO1" -pb 0 -gb 1 -ff $molargs2

# 6. delete file with command (in windows: del xxx, in linux: rm xxx)
del *md.* *run.* *formatted* result.dcd
```

- ✧ Please pay attention to the part "Calculate MM/GBSA" in the following:
  - ◆ -cs <complex selection: having the same rules as "atomselect" command in VMD>
  - ◆ -rs <receptor selection: having the same rules as "atomselect" command in VMD>
  - ◆ -ls <ligand selection: having the same rules as "atomselect" command in VMD>
  - ◆ Here, MolAICal supports recognizing commas "," in strings like "segname,A,C" as equivalent to spaces " ", meaning "segname,A,C" is treated identically to "segname A C".

- ✧ It is crucial to ensure that the selections for "-cs, -rs, and -ls" are correct. Users may try to understand the default segname naming rules in AutoPSF of VMD, for example: if it is the A

chain of a protein, the generated segname would be "AP1".

3.3. Calculate MM/PBSA based on the separate protein and ligand file in the Linux system

✧ **For APBS**, running the command below can calculate MM/PBSA for "protein.pdb" and "ligand.pdb" based on APBS:

```
#> molaical.exe -call run -c script -n runScript_linux_apbs.sh -i protein.pdb ligand.pdb ligand.str
```

Notice: The Linux version of APBS heavily depends on system library versions, so it may fail to run in some cases. When this happens, you can try different versions of APBS (Please see: https://github.com/Electrostatics/apbs). Once users find a suitable version, in the "runScript_linux_apbs.sh" file (for example, section 6 for MM/PBSA calculations), add "-pbe <Your APBS exe path>" at the end of the command line to directly specify the APBS executable path.

✧ **For DelPhi**

Delphi performs exceptionally well on different platforms. Even in Linux environments, it can run directly without relying on Linux system library files. However, Delphi has licensing restrictions—developers must apply for access through the official website to download it (http://compbio.clemson.edu/lab/delphisw). If the approval process takes time, as an emergency workaround, users may temporarily employ the Delphi executable files (named: "delphicpp_release") included in the official tutorial documents (http://compbio.clemson.edu/media/download/mmpbsa_example.zip).

Running the command below can calculate MM/PBSA for "protein.pdb" and "ligand.pdb" based on DelPhi:

```
#> molaical.exe -call run -c script -n runScript_linux_delphi.sh -i protein.pdb ligand.pdb ligand.str
```

The template script file "runScript_linux_delphi.sh" is as follows:

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs2 -dispdev text -args autopsf autopsf -mol 0 -top $molargs3 -prefix ligand
```

```
# 3. merge file
molaical.exe -call run -c merge -i -dispdev text -args -first ligand_formatted_autopsf.psf ligand_formatted_autopsf.pdb -second protein_formatted_autopsf.psf protein_formatted_autopsf.pdb -output complex_final

# 4. default to run 20 fs minimization and 40 fs MD simulation
molaical.exe -call run -c runnamd -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -nf md_configure.conf -output_prefix com_md

# 5. Cut last 10 steps for MM/PBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out result.dcd -first 51 -last 60 -stride 1 com_md.dcd

# 6. Calculate MM/PBSA
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -t result.dcd -cs "segname,AP1,CO1" -rs "segname,AP1" -ls "segname,CO1" -pb 1 -ff $molargs3

# 7. delete file with command (in windows: del xxx, in linux: rm xxx)
rm *md.* *run.* *formatted* result.dcd *final* *_m* *_wb*
```

- ✧ Please pay attention to the part "Calculate MM/PBSA" in the following:
  - ◆ -cs <complex selection: having the same rules as "atomselect" command in VMD>
  - ◆ -rs <receptor selection: having the same rules as "atomselect" command in VMD>
  - ◆ -ls <ligand selection: having the same rules as "atomselect" command in VMD>
  - ◆ Here, MolAICal supports recognizing commas "," in strings like "segname,A,C" as equivalent to spaces " ", meaning "segname,A,C" is treated identically to "segname A C".

- ✧ It is crucial to ensure that the selections for "-cs, -rs, and -ls" are correct. Users may try to understand the default segname naming rules in AutoPSF of VMD, for example: if it is the A chain of a protein, the generated segname would be "AP1".

3.4. Calculate MM/PBSA and MM/GBSA based on the separate protein and peptide in the Linux system

This tutorial covers the calculation of MM/GBSA and MM/PBSA for **proteins** and **peptides**. Unlike small molecule compounds, proteins and peptides **do not require additional force field parameters** since they consist of amino acids, allowing direct use of the default CHARMM force

field. This provides convenience for calculating the binding free energy of biological drugs.

The same methodology applies to MM/GBSA and MM/PBSA calculations for protein-protein complexes, protein-nucleic acid interactions, and other biomolecular systems.

Go to the tutorial directory:
#> cd 021-single_mmpbgbsa/peptide_protein

➢ **For MM/GBSA calculation of protein and peptide**, run the command below:

```
#> molaical.exe -call run -c script -n mmgbsa_linux.sh -i protein.pdb peptide.pdb PP1
```

The content of "mmgbsa_linux.sh" are below:

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs1 -dispdev text -args autopsf autopsf -mol 0 -prefix protein
molaical.exe -call run -c vmdargs -i -pdb $molargs2 -dispdev text -args autopsf autopsf -mol 0 -prefix ligand

# 3. merge file
molaical.exe -call run -c merge -i -dispdev text -args -first ligand_formatted_autopsf.psf ligand_formatted_autopsf.pdb -second protein_formatted_autopsf.psf protein_formatted_autopsf.pdb -output complex_final

# 4. run 2000 fs minimization
molaical.exe -call run -c runnamd -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -minimize 2000 -if_run 1 -output_prefix com

# 5. Cut last 10 steps for MM/GBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out result.dcd -first 1991 -last 2000 -stride 1 com_md.dcd

# 6. Calculate MM/GBSA
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -t result.dcd -cs "protein" -rs "not,segname,$molargs3" -ls "segname,$molargs3" -pb 0 -gb 1

# 7. delete file with command (in windows: del xxx, in linux: rm xxx)
rm *md.* *run.* *formatted* result.dcd
```

✧ "-if_run 1" represents that there are no MD simulations because it is hard to get a stable system phase.

✧ MolAICal defaults to providing molecular dynamics simulations with the Generalized Born Implicit Solvent model for MM/GBSA.

✧ Here, 2000 fs minimizations are carried out. This value needs to be evaluated by users. Here, the peptide and protein have a big system, so 2000 fs minimizations are required.

✧ Please pay attention to the part "Calculate MM/GBSA" in the following:

- -cs <complex selection: having the same rules as "atomselect" command in VMD>
- -rs <receptor selection: having the same rules as "atomselect" command in VMD>
- -ls <ligand selection: having the same rules as "atomselect" command in VMD>
- Here, MolAICal supports recognizing commas "," in strings like "segname,A,C" as equivalent to spaces " ", meaning "segname,A,C" is treated identically to "segname A C".

✧ It is crucial to ensure that the selections for "-cs, -rs, and -ls" are correct. Users may try to understand the default segname naming rules in AutoPSF of VMD, for example: if it is the A chain of a protein, the generated segname would be "AP1".

➢ **For MM/PBSA calculation of protein and peptide**, run the command below:

```
#> molaical.exe -call run -c script -n mmpbsa_linux_apbs.sh -i protein.pdb peptide.pdb PP1
```

The content of "mmpbsa_linux_apbs.sh" are below:

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs1 -dispdev text -args autopsf autopsf -mol 0 -prefix protein
molaical.exe -call run -c vmdargs -i -pdb $molargs2 -dispdev text -args autopsf autopsf -mol 0 -prefix ligand

# 3. merge file
molaical.exe -call run -c merge -i -dispdev text -args -first ligand_formatted_autopsf.psf ligand_formatted_autopsf.pdb -second protein_formatted_autopsf.psf protein_formatted_autopsf.pdb -output complex_final

# 4. default to run 60 fs minimization
molaical.exe -call run -c runnamd -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -nf md_configure.conf -output_prefix com_md

# 5. Cut last 10 steps for MM/PBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out result.dcd -first 51 -last 60 -stride 1 com_md.dcd

# 6. Calculate MM/PBSA
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -t result.dcd -cs "protein" -rs "not,segname,$molargs3" -ls "segname,$molargs3" -pb 2

# 7. delete file with command (in windows: del xxx, in linux: rm xxx)
rm *md.* *run.* *formatted* result.dcd *final.* *_md* *_wb*
```

✧ The file "md_configure.conf" is used as the input template configuration file of NAMD. MolAICal will automatically modify the necessary parameters of the input configuration file.

✧ Due to the absence of Generalized Born implicit solvent models in MM/PBSA calculations, a

60 fs minimization is sufficient here.

- ✧ Please pay attention to the part "Calculate MM/PBSA" in the following:
  - ◆ -cs <complex selection: having the same rules as "atomselect" command in VMD>
  - ◆ -rs <receptor selection: having the same rules as "atomselect" command in VMD>
  - ◆ -ls <ligand selection: having the same rules as "atomselect" command in VMD>
  - ◆ Here, MolAICal supports recognizing commas "," in strings like "segname,A,C" as equivalent to spaces " ", meaning "segname,A,C" is treated identically to "segname A C".

- ✧ It is crucial to ensure that the selections for "-cs, -rs, and -ls" are correct. Users may try to understand the default segname naming rules in AutoPSF of VMD, for example: if it is the A chain of a protein, the generated segname would be "AP1".

Appendix 1

Install external programs inside the MolAICal container (if finished, only for Linux Version MolAICal)

Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

To address the library dependency issues in Linux, the Linux version of MolAICal adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter

appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

```
The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";
```

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

3) Save and load the modified container (Option)

To preserve changes made within a container and avoid reinstalling software later, since all data will be lost if the container is deleted.

◆ The first step is to obtain the container ID and name:

```
#> molaical.exe -eset sys ps
```

◆ Secondly, clone this container as follows:

```
#> molaical.exe -eset sys export --clone -o molaicalv2.tar molaical
```

Note: 'molaical' is the container name. It can also be the container ID. If an error is reported, it may be due to permission issues with the mounted file system, which can be ignored.

◆ Thirdly, import the cloned container **if needing**:

```
#> molaical.exe -eset sys import --clone --name molaicalv2 molaicalv2.tar
```

◆ Now, change the loaded new container name to the default container name "molaical", the

previous container is renamed to "bakmolaical", the commands are as follows:

```
#> molaical.exe -eset sys rename molaical bakmolaical
#>molaical.exe -eset sys rename molaicalv2 molaical
```

Notice: Please note that a **container** (**dynamic**) is generated from an **image** (**static**). Operations such as software installation must be performed within the dynamic container; if cloning this container, restoring it will still be based on the original underlying image.

For more information, please see the manual of MolAICal or run "molaical.exe -eset sys --help" to get more help details.